
PROBABILISTIC FORECASTING VIA AUTOREGRESSIVE FLOW MATCHING

Anonymous authors
Anonymous submission

ABSTRACT

We propose FLOWTIME, a generative model for probabilistic forecasting of multivariate timeseries data whose empirical comparisons center on calibrated distributional accuracy and extrapolation. Given historical measurements and optional future covariates, we cast forecasting as sampling from a learned conditional distribution over future trajectories. The model decomposes the joint future distribution into a sequence of conditional densities, and each density is represented by a shared flow that transports a simple base distribution to the next-step observation distribution under the relevant covariates. We learn these transports with flow matching (FM), yielding a scalable, simulation-free training objective for the conditional transformations. This autoregressive FM factorization preserves the practical advantages of autoregressive forecasting—strong extrapolation behavior, compact parameterization, and calibrated uncertainty estimates—while retaining the ability of transport-based generative models to represent complex multi-modal conditional distributions. Across stochastic dynamical systems and real-world forecasting benchmarks, the reported CRPS and NRMSE comparisons show that FLOWTIME is consistently competitive and often best-performing, with especially large accuracy margins in extrapolation and on several real-world datasets.

1 INTRODUCTION

A central problem in modern machine learning is probabilistic timeseries forecasting: given observations up to the present, the goal is to quantify the distribution of plausible future system states. The quality of such forecasts is therefore judged not only by point accuracy, but also by uncertainty calibration, robustness across horizons, and performance under extrapolation, all of which are important in scientific, industrial, and societal applications (Lim & Zohren, 2021; Dama & Sinoquet, 2021; Ye et al., 2024).

Deep generative models have become a strong empirical route to this forecasting objective (Karl et al., 2016; Rasul et al., 2020; Desai et al., 2021). In this framing, a model is evaluated by the samples it draws from the future probability density conditioned on the past, and comparisons naturally focus on distributional scores such as CRPS as well as point-error metrics such as NRMSE. Diffusion and score-based generative models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2020) have produced state-of-the-art results on several forecasting benchmarks (Rasul et al., 2021; Tashiro et al., 2021; Kolloviah et al., 2023; Meijer & Chen, 2024), but those gains typically come with substantial computational cost during both training and inference.

Flow Matching (FM) (Liu et al., 2022; Albergo et al., 2023; Lipman et al., 2022) offers a generative-modeling framework that generalizes diffusion-style constructions while providing additional design flexibility. Instead of using iterative stochastic denoising over a discretized chain, FM learns deterministic probability paths that transform arbitrary base distributions into target distributions through continuous normalizing flows. This path-based formulation removes the need for handcrafted noise schedules and long ancestral sampling chains, making FM an attractive foundation when empirical comparisons must balance forecast quality against training and sampling efficiency.

Recent FM forecasting methods (Tamir et al., 2024; Hu et al., 2024; Kolloviah et al., 2024) report strong benchmark performance and improved computational efficiency relative to diffusion models. Their comparison structure, however, is usually window-level: the model learns the conditional

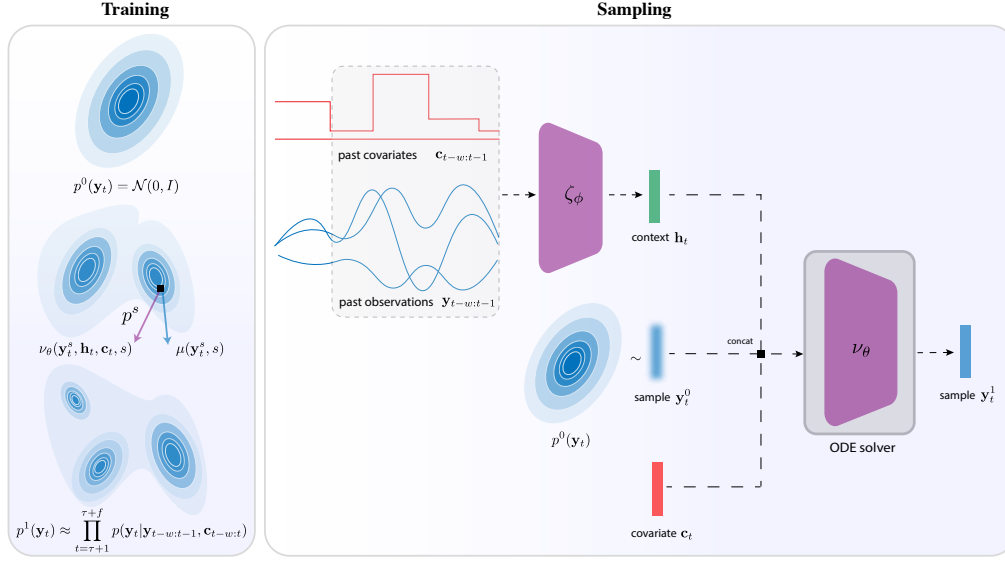


Figure 1: Training and sampling structure for FLOWTIME. During training, each next-step conditional comparison is represented by a probability path $(p^s)^{0 \leq s \leq 1}$ from the base distribution p^0 to the target distribution p^1 , generated by the velocity field μ . The learned network ν_θ regresses this velocity using the path sample $\mathbf{y}_t^s$ at flow step s , the context vector $\mathbf{h}_t$ summarizing past observations and covariates, and the current covariate $\mathbf{c}_t$. At inference time, samples are drawn from p^0 and transported by integrating the trained velocity field with an ODE solver until $s = 1$, producing the conditional forecast sample used in the empirical evaluations.

distribution of an entire fixed-horizon future block given a fixed-horizon context block. This non-autoregressive design can be fast, but the empirical trade-off is weaker extrapolation beyond the training horizon and less reliable uncertainty calibration. It also creates a harder optimization problem, because the velocity model must represent intricate time-dependent probability paths over the whole future window. The use of Gaussian-process-informed priors in (Kollovieh et al., 2024) addresses part of this optimization burden, but it also restricts the approach to univariate forecasting.

We introduce FLOWTIME as a direct alternative designed to make this empirical trade-off favorable in high-dimensional multivariate settings without informed priors. Rather than learning one joint distribution over a full future block, FLOWTIME factorizes forecasting into next-step conditional distributions, each conditioned on past observations and covariates. A shared flow transforms a simple base distribution into the next observation distribution, so the same learned mechanism is evaluated repeatedly across the forecast horizon. This autoregressive structure clarifies the comparison with non-autoregressive FM: it supports variable horizons, improves extrapolation beyond the training distribution, and produces calibrated uncertainty estimates while simplifying the conditional transport problem.

We evaluate FLOWTIME on classical stochastic dynamical systems and real-world forecasting datasets using CRPS for distributional quality and NRMSE for point accuracy. The resulting comparisons show consistent gains from the autoregressive factorization, including large NRMSE reductions against a non-autoregressive FM ablation on dynamical systems and top or near-top CRPS rankings across real-world benchmarks.

2 BACKGROUND: FLOW MATCHING

Flow matching (FM) (Lipman et al., 2022) provides the generative-modeling foundation for our forecasting comparisons because it enables continuous normalizing flows (CNFs) (Chen et al., 2018; Grathwohl et al., 2018) to be trained at large scale. CNFs represent transformations between distributions as solutions to ordinary differential equations (ODEs), giving invertible mappings with tractable likelihoods.

As in CNFs, FM learns a time-dependent diffeomorphic map on the data space Ω , denoted by the flow $\psi: [0, 1] \times \Omega \rightarrow \Omega$. The flow transports a source sample $\mathcal{X}^0 \sim p$ into a target sample $\mathcal{X}^1 := \psi(\mathcal{X}^0, 1)$ with $\mathcal{X}^1 \sim q$, thereby making the source-to-target comparison explicit. It is constructed by solving the following initial value problem:

$$d\psi(\mathcal{X}, s) = \mu(\psi(\mathcal{X}, s), s)ds, \quad \psi(\mathcal{X}, 0) = \mathcal{X}, \quad s \in [0, 1] \quad (1)$$

where $\mu: [0, 1] \times \Omega \rightarrow \Omega$ is the vector field that determines the flow velocity and induces the probability path $(p^s)_{0 \leq s \leq 1}$ over Ω , with $p^0 = p$. The modeling objective is to learn a valid μ whose terminal distribution satisfies $p^1 = q$.

In contrast to likelihood-trained CNFs, which require solving and differentiating through Eq. (1) during training, FM learns a parameterized vector field ν_θ by regression against vector fields associated with fixed conditional probability paths. This yields the tractable conditional flow matching (CFM) objective:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}[0, 1], \mathcal{X}^1 \sim q(\mathcal{X}^1), \mathcal{X} \sim p^s(\mathcal{X}|\mathcal{X}^1)} \|\nu_\theta(\mathcal{X}, s) - \mu(\mathcal{X}, s|\mathcal{X}^1)\|^2. \quad (2)$$

After minimizing this objective, samples from q are obtained by solving Eq. (1) with the trained ν_θ . Because training avoids gradient backpropagation through the ODE solver, FM scales stably to high-dimensional generative tasks such as image and video modeling (Esser et al., 2024; Polyak et al., 2024). The framework also accommodates multiple probability-path and conditioning choices (Liu et al., 2022; Albergo et al., 2023; Tong et al., 2023), which is what allows our experiments to isolate the effect of autoregressive versus non-autoregressive factorization.

3 AUTOREGRESSIVE FLOW MATCHING

Problem Setting Let $\mathbf{y}_\tau \in \mathbb{R}^n$ be an n -dimensional observation at time τ . From the l available measurements up to and including the present, $\mathbf{Y}_l = \{\mathbf{y}_{\tau-l}, \dots, \mathbf{y}_\tau\}$, we seek probabilistic forecasts for the next f values, $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$. Forecasts may also condition on covariates $\mathbf{C} = \{\mathbf{c}_{\tau-l}, \dots, \mathbf{c}_{\tau+f}\}$ with $\mathbf{c}_\tau \in \mathbb{R}^c$. Thus, each empirical evaluation asks whether the model can sample accurately from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ using training instances $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$.

We factorize this conditional distribution autoregressively across future time steps under a Markov assumption of order w :

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \quad (3)$$

where $w \leq l$ is the history size. The modeling target is therefore the next-step conditional distribution in Eq. (3), which is the unit on which the autoregressive and non-autoregressive comparisons differ. Rather than using maximum likelihood estimation as in many autoregressive models, we learn each conditional distribution with the flow matching framework (Lipman et al., 2022).

This factorization is designed to improve the empirical comparison along several dimensions. First, it replaces one high-dimensional future-window distribution with simpler conditional distributions $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$. Second, the conditionals can be trained with teacher forcing, so all time steps contribute training signal in parallel. Third, the Markov window w controls the capacity-computation trade-off. Finally, because uncertainty is propagated through an explicit chain of conditionals, the model exposes how forecast uncertainty evolves across prediction and extrapolation horizons.

3.1 CONSTRUCTING A TARGET PROBABILITY PATH

For each future time step $t \in [\tau + 1, \tau + f]$, we define a target probability path $(p^s(\mathbf{y}_t))_{0 \leq s \leq 1}$ whose endpoints align the base and empirical conditional distributions: $p^0(\mathbf{y}_t) = \mathcal{N}(0, I)$ and $p^1(\mathbf{y}_t) \approx \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \approx p_{\mathcal{D}}$, where $p_{\mathcal{D}}$ denotes the empirical distribution. This makes each forecast step a comparable transport problem. The path is generated along s by a flow field $\psi: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ defined through the ordinary differential equation (ODE):

$$d\psi(\mathbf{y}_t, s) = \mu(\psi(\mathbf{y}_t, s), s)ds, \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t \quad s \in [0, 1] \quad (4)$$

where $\mu: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the vector field that constructs the flow and is chosen so that the resulting path satisfies the continuity equation.

To obtain a tractable objective for this per-step transport, we express the marginal probability path as a mixture of conditional probability paths:

$$p^s(\mathbf{y}_t) = \int p^s(\mathbf{y}_t|\mathbf{z})p(\mathbf{z})d\mathbf{z} \quad (5)$$

where $\mathbf{z} \sim \pi_{0,1}(\mathbf{z})$ is a conditioning random variable sampled from an arbitrary data coupling $\pi_{0,1}$. Theorem 2 in (Lipman et al., 2022) shows that the conditional velocity fields $\mu(\mathbf{y}_t, s|\mathbf{z})$ associated with $p^s(\mathbf{y}_t|\mathbf{z})$ recover the corresponding marginal velocity field $\mu(\mathbf{y}_t, s)$, so the regression target remains valid for the unconditional path.

Following (Tong et al., 2023), we choose $\mathbf{z} = \{\mathbf{y}_t^0, \mathbf{y}_t^1\}$ with $p(\mathbf{z} = (\mathbf{y}_t^0, \mathbf{y}_t^1)) = p^0 \times p^1$, where $\mathbf{y}_t^0$ and $\mathbf{y}_t^1$ are samples from p^0 and p^1 , respectively. This yields the following conditional probability path:

$$p^s(\mathbf{y}_t|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I) \quad (6)$$

with marginals satisfying $p^0(\mathbf{y}_t|\mathbf{z}) = p^0(\mathbf{y}_t)$ and $p^1(\mathbf{y}_t|\mathbf{z}) = p^1(\mathbf{y}_t)$ for $\sigma^2 \rightarrow 0$. The corresponding conditional velocity field is the simple difference $\mu(\mathbf{y}_t, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$. This straight Euclidean path provides closed-form training targets, which keeps the empirical gains attributable to the autoregressive factorization rather than to a specialized path construction. Once the velocity field is learned, samples from $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ are obtained by solving the ODE in Eq. (4) until $s = 1$.

3.2 TRAINING

As in standard FM, we learn the velocity field by regressing it with a neural network ν_θ parameterized by θ under the conditional flow matching objective in Eq. 2. For the forecasting comparison, the conditioning information is summarized by a context vector $\mathbf{h}_t \in \mathbb{R}^h$ for each time t :

$$\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1}) \quad (7)$$

where $\zeta: \mathbb{R}^{w \times n} \times \mathbb{R}^{w \times c} \rightarrow \mathbb{R}^h$ is a neural network with parameters ϕ . The resulting learning objective is:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{y}_t^s \sim p^s(\mathbf{y}_t|\mathbf{z})} \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2 \quad (8)$$

We optimize this objective by sampling a time point t , its context window, and the associated covariates from $\mathcal{D}$, then jointly updating ϕ and θ by stochastic gradient descent. This training regime is analogous to teacher forcing (Williams & Zipser, 1989): ground-truth histories, rather than model-generated histories, condition each training example. The design allows parallel training across time steps while reserving error-accumulation behavior for the autoregressive sampling evaluation. Algorithm 1 summarizes the training procedure.

Algorithm 1 Training: Autoregressive Flow Matching.

Require: Dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, window size w , networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
 - 2: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$ and $(\mathbf{y}_t^1, \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \sim \mathcal{D}$
 - 3: Set $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$
 - 4: Sample $s \sim \mathcal{U}(0, 1)$ and $\mathbf{y}_t^s \sim \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I)$
 - 5: Compute target: $\mu(\mathbf{y}_t^s, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$
 - 6: Compute loss: $\mathcal{L}_t = \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2$
 - 7: Update θ, ϕ
 - 8: **end while**
 - 9: **return** ν_θ, ζ_ϕ
-

3.3 INFERENCE

After training, inference produces probabilistic forecasts for new observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$ over the next f timesteps. We use the rolling-window sampling procedure in Algorithm 2: each sampled next-step value is appended to the context before the following step is generated. Repeating this procedure yields empirical predictive quantiles, which are the samples used to compute distributional metrics such as CRPS and to visualize forecast intervals.

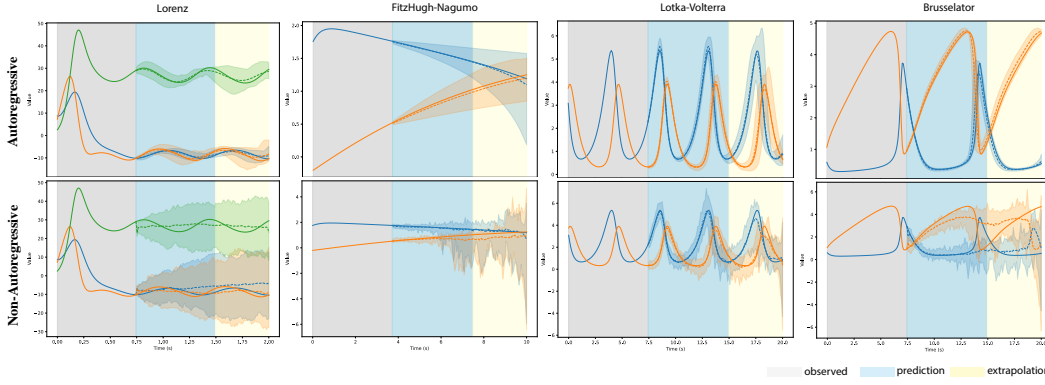


Figure 2: Qualitative forecast comparison between autoregressive flow matching and standard flow matching on four stochastic dynamical systems. Solid curves show ground truth; dashed curves show the predictive mean with the 95% confidence interval. Each panel separates the in-distribution prediction regime from the extrapolation regime, highlighting the horizon-wise behavior that is quantified by CRPS and NRMSE in Table 1.

Algorithm 2 Sampling: Autoregressive Flow Matching.

Require: Past observations $\mathbf{Y}_l$, covariates $\mathbf{C}$, forecast horizon f , window size w , networks ν_θ, ζ_ϕ

- 1: Initialize $\mathbf{Y}_f \leftarrow \mathbf{Y}_l$
- 2: **for** $t = \tau + 1$ **to** $\tau + f$ **do**
- 3: Set context: $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$, using the last w entries in $\mathbf{Y}_f$
- 4: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$
- 5: Solve $\frac{d\psi(\mathbf{y}_t, s)}{ds} = \nu_\theta(\psi(\mathbf{y}_t, s), \mathbf{h}_t, \mathbf{c}_t, s)$, $\psi(\mathbf{y}_t, 0) = \mathbf{y}_t^0$, $s \in [0, 1]$
- 6: Append $\psi(\mathbf{y}_t, 1)$ to $\mathbf{Y}_f$
- 7: **end for**
- 8: **return** $\mathbf{Y}_f$

4 EXPERIMENTS

This section reports the empirical comparisons used to evaluate FLOWTIME. We separate controlled stochastic dynamical-system experiments, where the main comparison is autoregressive versus non-autoregressive FM under prediction and extrapolation conditions, from real-world forecasting benchmarks, where FLOWTIME is compared with statistical, deep autoregressive, diffusion, and flow-matching baselines.

4.1 DYNAMICAL SYSTEMS

Data We evaluated FLOWTIME on five stochastic dynamical systems that provide controlled tests of distributional forecasting and extrapolation: stochastic versions of the Lorenz system (Lorenz, 1963), FitzHugh–Nagumo model (FitzHugh, 1961), Lotka–Volterra system Lotka (1925), Brusselator (Lefever & Prigogine, 1968), and Van der Pol oscillator (van der Pol, 1926). For each system, trajectories were generated by numerically solving the corresponding stochastic differential equations with randomly sampled initial conditions and Brownian motion realizations. Appendix A.1 specifies the systems, trajectory counts, and train/test partition used for these quantitative comparisons.

Implementation details To keep the autoregressive and non-autoregressive comparison focused on factorization rather than model scale, we used a compact architecture for the dynamical-system experiments. The context encoder ζ_ϕ was a two-layer bi-directional LSTM with 64 hidden units, and the velocity network ν_θ was an MLP with three hidden layers of size 64. Fourier positional embeddings encoded the flow step s as a 16-dimensional vector with fixed frequencies, and the context length w was set equal to the prediction length. Models were trained with stochastic gradient

descent using Adam, learning rate 0.003, and batch size 128; convergence required ~ 7 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines The primary dynamical-system baseline is a non-autoregressive flow-matching model that directly learns the joint future-window distribution conditioned on past observations and covariates, i.e. $p^1 = p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C})$. This baseline is a direct ablation of the FLOWTIME factorization: both methods use FM, but only FLOWTIME decomposes the future into autoregressive conditionals. The non-autoregressive formulation is closely related to (Kollovieh et al., 2024), and Appendix B gives its implementation details.

Evaluation Each trajectory was split into three ordered segments so that prediction and extrapolation could be reported separately: (1) an observation window used as historical input, (2) a prediction window used as the training target, and (3) an extrapolation window that was never observed during training. We report continuous ranked probability score (CRPS) (Winkler et al., 1996) to compare predictive distributions and normalized root mean square error (NRMSE) (Hyndman & Koehler, 2006) to compare point accuracy. Lower values are better for both metrics, so the table directly reports the margins between the autoregressive and non-autoregressive factorizations.

Table 1: Autoregressive (FLOWTIME) versus non-autoregressive (Non-AR) FM on five stochastic dynamical systems. Test-set CRPS evaluates distributional forecast quality and NRMSE evaluates point accuracy, with both metrics reported separately for the training-horizon prediction regime and the unseen extrapolation regime. Values are mean and standard deviation across five random seeds; lower is better, and bold marks the better factorization for each system, metric, and regime.

System	Factorization	Prediction		Extrapolation	
		CRPS ↓	NRMSE ↓	CRPS ↓	NRMSE ↓
Lorenz	Non-AR	0.147±0.012	0.242±0.018	0.120±0.009	0.301±0.024
	FLOWTIME	0.090±0.008	0.017±0.003	0.137±0.011	0.048±0.006
FitzHugh-Nagumo	Non-AR	0.165±0.014	0.131±0.011	0.214±0.016	0.306±0.022
	FLOWTIME	0.144±0.013	0.090±0.007	0.146±0.012	0.278±0.019
Lotka-Volterra	Non-AR	0.130±0.011	0.266±0.019	0.136±0.012	0.502±0.033
	FLOWTIME	0.126±0.010	0.189±0.015	0.123±0.009	0.339±0.026
Brusselator	Non-AR	0.217±0.016	0.658±0.042	0.238±0.019	1.046±0.057
	FLOWTIME	0.125±0.011	0.031±0.004	0.109±0.008	0.023±0.003
Van der Pol	Non-AR	0.143±0.012	0.161±0.014	0.211±0.018	0.510±0.031
	FLOWTIME	0.225±0.017	0.051±0.005	0.282±0.021	0.101±0.009

Results Table 1 shows a clear and consistent accuracy advantage for the autoregressive factorization. In the prediction regime, FLOWTIME obtains the lower NRMSE on all five systems, with the largest margins on Lorenz (0.017 vs 0.242) and Brusselator (0.031 vs 0.658), corresponding to 93% and 95% lower error than the Non-AR baseline. Distributional quality follows the same pattern in most cases: FLOWTIME achieves the lower CRPS on four of five systems, with Van der Pol the only prediction-regime CRPS exception.

The extrapolation results strengthen the practical significance of the factorization. FLOWTIME retains the lower NRMSE on every dynamical system after the horizon moves beyond the training target window, including a 98% reduction on Brusselator (0.023 vs 1.046). It also achieves lower extrapolation CRPS on FitzHugh-Nagumo, Lotka-Volterra, and Brusselator, while Non-AR has the lower CRPS on Lorenz and Van der Pol. Overall, the table indicates that the autoregressive FM design is especially reliable for point accuracy and remains strongly competitive for calibrated uncertainty when the evaluation shifts from prediction to extrapolation.

4.2 REAL-WORLD DATASETS

Data We next evaluated FLOWTIME on real-world GluonTS forecasting tasks (Alexandrov et al., 2020) spanning univariate and multivariate series at hourly and daily frequencies. The benchmark set consists of Electricity (Lai et al., 2018), Solar (Lai et al., 2018), Exchange (Dua et al., 2017), Traffic (Dua et al., 2017), and Wikipedia (Gasthaus et al., 2019). Appendix A.2 summarizes the dataset dimensions, domains, frequencies, training lengths, and prediction horizons used for the CRPS comparisons.

Implementation details For the real-world benchmarks, ζ_ϕ was a three-layer bi-directional LSTM with 64 hidden units. The velocity network ν_θ was a residual neural network with five residual blocks implemented by 1-D convolutional layers and gated activations (Van Den Oord et al., 2016; Kong et al., 2020). This architecture follows the design in (Rasul et al., 2021) while using fewer residual blocks. Fourier positional embeddings encoded the flow step s into a 32-dimensional vector with fixed frequencies, and the same embedding strategy was used for time-dependent covariates c . Models were trained with Adam using learning rate 0.001 and batch size 128; depending on the dataset, convergence took 40 – 100 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines We compared FLOWTIME with established statistical, deep learning, diffusion, and flow-matching baselines. The statistical group includes Seasonal Naive (SN), AutoARIMA, and AutoETS (Hyndman et al., 2008); the deep autoregressive group includes DeepAR (Salinas et al., 2020) and WaveNet (Van Den Oord et al., 2016). We also include diffusion baselines TSDiff (Kollovich et al., 2023), SSSD (Alcaraz & Strodthoff, 2022), and TimeGrad (Rasul et al., 2021), together with the FM baseline TSFlow (Kollovich et al., 2024). This comparison places FLOWTIME against both classical forecasting methods and recent generative forecasting models.

Table 2: Real-world test-set CRPS comparison across five forecasting datasets; lower is better. FLOWTIME entries report mean and standard error over five re-training and evaluation runs, while baseline values are taken from (Kollovich et al., 2024). Best scores are in **bold** and second-best scores are underlined, making the dataset-wise ranking and margins explicit.

Method	Electr.	Exchange	Solar	Traffic	Wikipedia
SN	0.060±0.000	0.013±0.000	0.512±0.000	0.221±0.000	0.423±0.000
ARIMA	0.344±0.000	0.013±0.000	0.558±0.003	0.486±0.000	0.654±0.000
ETS	0.056±0.000	0.008±0.000	0.550±0.000	0.492±0.000	0.651±0.000
DeepAR	0.051±0.000	0.013±0.004	0.429±0.015	0.103±0.000	0.215±0.003
WaveNet	0.058±0.008	0.012±0.001	0.360±0.000	0.099±0.000	0.207 ±0.000
CSDI	0.051±0.000	0.013±0.000	0.360±0.000	0.152±0.000	0.318±0.012
SSSD	0.048±0.000	0.010±0.000	0.354±0.024	0.107±0.002	0.209±0.000
TSFlow	0.045±0.001	0.009 ±0.001	0.343±0.002	0.083 ±0.000	0.227±0.000
FLOWTIME	0.042 ±0.001	0.009 ±0.001	0.284 ±0.002	<u>0.089</u> ±0.000	0.243±0.002

Results Table 2 reports a dataset-wise CRPS comparison on five real-world benchmarks. FLOWTIME is best or tied for best on three datasets: Electricity, Exchange, and Solar. On Electricity, it achieves the lowest CRPS, 0.042, improving over the second-best TSFlow result of 0.045 by 6.7%. On Exchange, FLOWTIME matches the best reported score, 0.009, tying TSFlow while remaining ahead of the other statistical and deep learning baselines. The largest real-world margin appears on Solar, where FLOWTIME obtains CRPS 0.284 compared with TSFlow’s second-best 0.343, a 17.2% error reduction.

The remaining datasets show that the method is still competitive when it is not the top-ranked entry. On Traffic, FLOWTIME ranks second with CRPS 0.089, behind TSFlow at 0.083 and ahead of all other listed statistical, autoregressive, and diffusion baselines. On Wikipedia, FLOWTIME reaches CRPS 0.243, behind WaveNet (0.207), SSSD (0.209), and TSFlow (0.227), but still substantially better than the traditional baselines SN, ARIMA, and ETS. Taken together, the table shows that the autoregressive FM design delivers leading performance on most of the real-world benchmark set while maintaining competitive CRPS across diverse frequencies and dimensionalities.

5 DISCUSSION

We introduced FLOWTIME, a probabilistic forecasting model that combines autoregressive modeling with flow matching to produce multivariate forecast samples with clear distributional and point-accuracy evaluation targets. By replacing a single full-window conditional distribution with a sequence of next-step conditional transports, FLOWTIME aligns the structure of the model with the way forecast quality is measured across horizons. The empirical results show that this factorization produces favorable and interpretable margins. Against the non-autoregressive FM ablation on stochastic dynamical systems, FLOWTIME achieves lower prediction NRMSE on all five systems and lower extrapolation NRMSE on all five systems, with the largest reported reductions reaching 95% in prediction and 98% in extrapolation on Brusselator. On real-world datasets, FLOWTIME is best or tied for best on three of five CRPS benchmarks and remains second-best on Traffic. These patterns indicate that the autoregressive decomposition improves accuracy and remains competitive for uncertainty calibration, particularly when evaluation moves beyond the training horizon.

The same comparison also clarifies the method’s limitations. Autoregressive sampling is sequential, so inference has more overhead than fully parallel non-autoregressive generation, which may matter in real-time forecasting systems. In addition, the present implementation applies flow matching directly in data space, a choice that may be less efficient for high-dimensional partially observed data. Future work could test whether latent-space formulations preserve the reported CRPS and NRMSE advantages, extend the approach to irregularly sampled data, and evaluate broader scientific and decision-making applications. Overall, the results support FLOWTIME as a practical flow-based forecasting model whose strongest empirical evidence is consistent accuracy and favorable extrapolation behavior.

REFERENCES

- Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.
- Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and forecasting with structured state space models. *arXiv preprint arXiv:2208.09399*, 2022.
- Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, et al. GluonTS: Probabilistic and neural time series modeling in python. *Journal of Machine Learning Research*, 21(116):1–6, 2020.
- Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
- Fatoumata Dama and Christine Sinoquet. Time series analysis and modeling to forecast: a survey. In *ArXiv:2104.00164*, 2021.
- Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. TimeVAE: A variational auto-encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095*, 2021.
- Dheeru Dua, Casey Graff, et al. UCI machine learning repository. URL <http://archive.ics.uci.edu/ml>, 7(1):62, 2017.
- Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024.
- Richard FitzHugh. Impulses and physiological states in theoretical models of nerve membrane. *Biophysical Journal*, 1(6):445–466, 1961. doi: 10.1016/S0006-3495(61)86902-6.
- Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas, Valentin Flunkert, and Tim Januschowski. Probabilistic forecasting with spline quantile function RNNs. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 1901–1910. PMLR, 2019.

-
- Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD: Free-Form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2018.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.
- Yang Hu, Xiao Wang, Lirong Wu, Huatian Zhang, Stan Z. Li, Sheng Wang, and Tianlong Chen. Fm-ts: Flow matching for time series generation. In *ArXiv:2411.07506v1 [cs.LG]*, 2024. URL <http://arxiv.org/abs/2411.07506>.
- Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with Exponential Smoothing: The State Space Approach*. Springer Science & Business Media, 2008.
- Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.
- Maximilian Karl, Maximilian Soelch, Justin Bayer, and Patrick Van der Smagt. Deep variational Bayes filters: Unsupervised learning of state space models from raw data. *arXiv preprint arXiv:1605.06432*, 2016.
- Marcel Kollovich, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang, and Yuyang Bernie Wang. Predict, refine, synthesize: Self-guiding diffusion models for probabilistic time series forecasting. *Advances in Neural Information Processing Systems*, 36:28341–28364, 2023.
- Marcel Kollovich, Marten Lienen, David Lüdke, Leo Schwinn, and Stephan Günnemann. Flow matching with gaussian process priors for probabilistic time series forecasting. In *The Thirteenth International Conference on Learning Representations*, 2024.
- Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. DiffWave: A versatile diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.
- Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term temporal patterns with deep neural networks. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*, pp. 95–104, 2018.
- R. Lefever and I. Prigogine. Symmetry breaking instabilities in dissipative systems. II. *The Journal of Chemical Physics*, 48(4):1695–1700, 1968. doi: 10.1063/1.1668896.
- Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical Transactions of the Royal Society A*, 379(2194):20200209, 2021.
- Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *The Eleventh International Conference on Learning Representations*, 2022.
- Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2022.
- Edward Lorenz. Deterministic nonperiodic flow. *Journal of Atmospheric Sciences*, 20(2), 1963.
- Alfred J. Lotka. *Elements of Physical Biology*. Williams & Wilkins, 1925.
- Caspar Meijer and Lydia Y Chen. The rise of diffusion models in time-series forecasting. *arXiv preprint arXiv:2401.03006*, 2024.
- Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. MovieGen: A cast of media foundation models. *arXiv e-prints*, pp. arXiv–2410, 2024.
- Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. Multivariate probabilistic time series forecasting via conditioned normalizing flows. *arXiv preprint arXiv:2002.06103*, 2020.

486 Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising dif-
487 fusion models for multivariate probabilistic time series forecasting. In *International Conference*
488 *on Machine Learning*, pp. 8857–8868. PMLR, 2021.

489 David Salinas, Michael Bohlke-Schneider, Laurent Callot, Roberto Medico, and Jan Gasthaus.
490 High-dimensional multivariate forecasting with low-rank gaussian copula processes. *Advances*
491 *in Neural Information Processing Systems*, 32, 2019.

492 David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic
493 forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):
494 1181–1191, 2020.

495 Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised
496 learning using nonequilibrium thermodynamics. In *International Conference on Machine Learn-*
497 *ing*, pp. 2256–2265. PMLR, 2015.

498 Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben
499 Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint*
500 *arXiv:2011.13456*, 2020.

501 Ella Tamir, Najwa Laabid, Markus Heinonen, Vikas Garg, and Arno Solin. Conditional flow match-
502 ing for time series modelling. In *ICML Workshop: Structured Probabilistic Inference and Gener-*
503 *ative Modeling*, 2024.

504 Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based
505 diffusion models for probabilistic time series imputation. *Advances in Neural Information Pro-*
506 *cessing Systems*, 34:24804–24816, 2021.

507 Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Hugué, Yanlei Zhang, Jarrid Rector-
508 Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models
509 with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.

510 Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves,
511 Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. WaveNet: A generative model for
512 raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016.

513 B. van der Pol. On ‘relaxation-oscillations’. *The London, Edinburgh, and Dublin Philosophical*
514 *Magazine and Journal of Science*, 2(11):978–992, 1926. doi: 10.1080/14786442608564127.

515 Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent
516 neural networks. *Neural Computation*, 1(2):270–280, 1989.

517 Robert L Winkler, Javier Munoz, José L Cervera, José M Bernardo, Gail Blattenberger, Joseph B
518 Kadane, Dennis V Lindley, Allan H Murphy, Robert M Oliver, and David Ríos-Insua. Scoring
519 rules and the evaluation of probabilities. *Test*, 5:1–60, 1996.

520 Jiexia Ye, Weiqi Zhang, Ke Yi, Yongzi Yu, Ziyue Li, Jia Li, and Fugee Tsung. A survey of time
521 series foundation models: Generalizing time series representation with large language model. In
522 *ArXiv:2104.00164*, 2024.

A DATASETS

A.1 DYNAMICAL SYSTEMS

For the controlled experiments, we generated train and test trajectories for five classical stochastic dynamical systems—Lorenz, FitzHugh-Nagumo, Lotka-Volterra, Brusselator, and Van der Pol—by numerically integrating their stochastic differential equations (SDEs). Each benchmark system follows the general form

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t; \Theta) dt + \sigma d\mathbf{W}(t),$$

where f is the deterministic drift, Θ are the system parameters, σ is a constant diffusion term, and $\mathbf{W}(t)$ is standard Brownian motion. For each system, we sampled 2400 random initial conditions from the uniform range specified in Table 3 and generated one trajectory per initial condition using the Euler-Heun method with fixed step size. Each trajectory contains 200 equally spaced time points, split into 75 observed points, 75 prediction points, and 50 extrapolation points. The final quantitative comparison uses 2000 trajectories for training and 400 trajectories for testing.

Table 3: Specification of the five stochastic dynamical systems used for the controlled quantitative comparison. $\mathbf{f}(\mathbf{x})$ is the drift function, Θ lists the system parameters, σ is the constant diffusion term, and $p(\mathbf{x}(0))$ is the initial-condition distribution. Each system is integrated over $[t_0, t_1]$ with 200 Euler-Heun steps, yielding the observed, prediction, and extrapolation segments used in Table 1.

System	$f(\mathbf{x})$	Θ	Σ	$p(\mathbf{x}(0))$	$[t_0, t_1]$
Lorenz	$f_1 = \sigma(x_2 - x_1)$	$\sigma = 10$	(1.5, 1.5, 1.5)	$\mathcal{U}([0, 10])$	[0, 2]
	$f_2 = x_1(\rho - x_3) - x_2$	$\rho = 28$			
	$f_3 = x_1 x_2 - \beta x_3$	$\beta = \frac{8}{3}$			
FitzHugh-Nagumo	$f_1 = x_1 - \frac{x_1^3}{3} - x_2 + I$	$a = 0.7$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 10]
	$f_2 = \frac{x_1 + a - b x_2}{\tau}$	$b = 0.8$			
		$\tau = 12.5$			
Lotka-Volterra		$\alpha = 1.3$	(1.5, 1.5)	$\mathcal{U}([0, 5])$	[0, 20]
	$f_1 = \alpha x_1 - \beta x_1 x_2$	$\beta = 0.9$			
	$f_2 = -\delta x_2 + \gamma x_1 x_2$	$\gamma = 0.8$			
Brusselator		$\delta = 1.8$	(1.5, 1.5)	$\mathcal{U}([0, 2])$	[0, 20]
	$f_1 = A + x_1^2 x_2 - (B + 1) x_1$	$A = 1.0$			
	$f_2 = B x_1 - x_1^2 x_2$	$B = 3.0$			
Van der Pol	$f_1 = x_2$	$\mu = 0.1$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 20]
	$f_2 = \mu(1 - x_1^2) x_2 - x_1$				

A.2 REAL-WORLD DATASETS

We used five GluonTS datasets (Alexandrov et al., 2020) to evaluate real-world forecasting performance, using the versions preprocessed as in (Salinas et al., 2019). Table 4 records the dataset characteristics needed to contextualize the reported CRPS values, including dimensionality, domain, frequency, training length, and forecast horizon.

Table 4: Real-world dataset properties used to interpret the CRPS comparisons: dimension n , data domain Ω , sampling frequency, total training timesteps, and prediction length.

Dataset	n	Ω	freq.	timesteps	prediction length
Electricity	370	$\mathbb{R}^+$	hourly	5833	24
Exchange	8	$\mathbb{R}^+$	daily	6071	30
Solar	137	$\mathbb{R}^+$	hourly	7009	24
Traffic	963	(0, 1)	hourly	4001	24
Wikipedia	2000	$\mathbb{N}$	daily	792	30

B NON-AUTOREGRESSIVE FLOW MATCHING BASELINE

Problem Setting The non-autoregressive baseline uses the same forecasting inputs as Section 3: past observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$ define the target conditional distribution $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. The difference is the factorization being tested. Instead of decomposing the future into next-step conditionals, the baseline directly models the entire future trajectory $\mathbf{Y}_f \in \mathbb{R}^{f \times n}$ as one joint random variable:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = p(\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}|\mathbf{Y}_l, \mathbf{C}) \quad (9)$$

This formulation preserves temporal correlations across all future time steps, but the quantitative comparison in Table 1 tests the cost of learning this higher-dimensional distribution without autoregressive structure.

Training The baseline constructs a trajectory-level probability path $(p^s(\mathbf{Y}_f))^{0 \leq s \leq 1}$ that transports samples from the prior $p^0(\mathbf{Y}_f) = \mathcal{N}(0, \Sigma)$ to the target $p^1(\mathbf{Y}_f) \approx p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$, where Σ is block diagonal. The prior can be interpreted as independent Brownian motion processes per dimension, with Brownian motion defined as a stochastic process $W: [\tau + 1, \tau + f] \rightarrow \mathbb{R}^n$.

The conditional probability path is a linear interpolation bridge with Brownian noise:

$$p^s(\mathbf{Y}_f|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma) \quad (10)$$

where $\mathbf{z} = (\mathbf{Y}_f^0, \mathbf{Y}_f^1)$ with $\mathbf{Y}_f^0 \sim p^0$ and $\mathbf{Y}_f^1 \sim p^1$. The corresponding conditional velocity field is:

$$\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0 + \frac{\sigma^2(1-2s)}{2}\Sigma^{-1}(\mathbf{Y}_f - ((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1)) \quad (11)$$

When $\sigma^2 \rightarrow 0$, this reduces to the straight path velocity $\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0$. The neural velocity field ν_θ therefore operates on the full future trajectory rather than on one forecast step. Its context encoder remains analogous to Eq. (7):

$$\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f}) \quad (12)$$

where ζ_ϕ now processes both past and future covariates. The resulting training objective is:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{Y}_f^s \sim p^s(\mathbf{Y}_f|\mathbf{z})} \|\mu(\mathbf{Y}_f^s, s|\mathbf{z}) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|^2 \quad (13)$$

where ν_θ is implemented as a sequential neural network (e.g., Transformer or RNN) that processes the full trajectory at once. Sampling consequently solves a trajectory-level ODE:

$$\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s), \quad \psi(\mathbf{Y}_f, 0) \sim p^0 \quad (14)$$

using numerical solvers. The output is a joint sample from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ generated without autoregressive decomposition, enabling the direct factorization ablation.

Implementation Details We used this baseline for the dynamical-system forecasting ablation. In that setting, ν_θ was a 4-layer bi-directional LSTM with 128 hidden units, and the training hyperparameters matched Section 4.1. Training to convergence took ~ 2 minutes on a single A100-SXM4-40GB NVIDIA GPU, making the comparison favorable to the baseline in raw training time while still allowing Table 1 to assess forecast quality.

Key Differences from Autoregressive Variant The non-autoregressive baseline differs from FLOWTIME in the exact modeling choice tested by the ablation. FLOWTIME uses a Markovian autoregressive decomposition over future time steps, whereas the baseline models $\mathbf{Y}_f$ as a single high-dimensional random variable. This preserves cross-temporal dependencies directly, but it also requires learning a distribution over $\mathbb{R}^{n \times f}$ rather than a sequence of lower-dimensional conditionals. The computational and uncertainty-propagation profiles also differ. The non-autoregressive model processes whole trajectories through the velocity network and can generate all future time points jointly, at the cost of higher memory use. FLOWTIME samples sequentially, but the chain of conditional distributions makes error accumulation and horizon-wise uncertainty explicit. Thus the comparison is not only between two implementations, but between two empirical trade-offs: joint trajectory modeling with parallel generation versus autoregressive conditional modeling with stronger observed NRMSE and extrapolation behavior in Table 1.

Algorithm 3 Non-Autoregressive Flow Matching: Training

Require: Dataset $\mathcal{D}$, networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$ and $(\mathbf{Y}_f^1, \mathbf{Y}_l, \mathbf{C}) \sim \mathcal{D}$
- 3: Set $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$
- 5: $\mathbf{Y}_f^s \sim \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma)$
- 6: Compute target $\mu(\mathbf{Y}_f^s, s|\mathbf{z})$
- 7: Update θ, ϕ to minimize $\|\mu(\cdot) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|_{\Sigma^{-1}}^2$
- 8: **end while**

Algorithm 4 Non-Autoregressive Flow Matching: Sampling

Require: $\mathbf{Y}_l, \mathbf{C}$, networks ν_θ, ζ_ϕ

- 1: Encode context: $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C})$
- 2: Sample initial trajectory $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$
- 3: Solve $\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)$
- 4: Return $\psi(\mathbf{Y}_f, 1)$

C EVALUATION METRICS

This section defines the two metrics used throughout the empirical comparisons. Given an n -dimensional forecast horizon $\mathbf{Y}_f = \{y_{\tau+1}, \dots, y_{\tau+f}\}$ and predictive samples from the probabilistic model, we report Continuous Ranked Probability Score (CRPS) for distributional quality and Normalized Root Mean Squared Error (NRMSE) for point accuracy. In both cases, lower values indicate better forecasts.

C.1 CONTINUOUS RANKED PROBABILITY SCORE (CRPS)

CRPS is the primary metric for comparing predictive distributions because it rewards both calibration and sharpness. For a univariate random variable X with cumulative distribution function (CDF) F_X and observed value x , the CRPS is defined as:

$$\text{CRPS}(F_X, x) = \int_{-\infty}^{+\infty} (F_X(y) - \mathbf{1}\{y \geq x\})^2 dy, \quad (15)$$

where $\mathbf{1}\{\cdot\}$ is the indicator function. The score compares the full predictive CDF with the realized observation, so lower CRPS values correspond to better-calibrated and more accurate probabilistic forecasts.

For time series forecasting, CRPS is computed at each forecast horizon and then averaged so that methods can be compared with a single distributional score:

$$\text{CRPS}_{\text{avg}} = \frac{1}{f} \sum_{k=1}^f \text{CRPS}(F_{X_{\tau+k}}, y_{\tau+k}), \quad (16)$$

where $F_{X_{\tau+k}}$ is the predicted CDF, or the empirical CDF estimated from samples, at horizon k , and $y_{\tau+k}$ is the corresponding ground-truth observation.

C.2 NORMALIZED ROOT MEAN SQUARED ERROR (NRMSE)

NRMSE complements CRPS by measuring point-forecast accuracy on a scale-normalized basis. For predictions $\{\hat{y}_{\tau+1}, \dots, \hat{y}_{\tau+f}\}$ and ground-truth values $\{y_{\tau+1}, \dots, y_{\tau+f}\}$, the unnormalized RMSE is:

$$\text{RMSE} = \sqrt{\frac{1}{f} \sum_{k=1}^f (\hat{y}_{\tau+k} - y_{\tau+k})^2}. \quad (17)$$

To make this error comparable across systems and datasets with different magnitudes, we normalize RMSE by the standard deviation of the ground-truth series, σ_y :

$$\text{NRMSE} = \frac{\text{RMSE}}{\sigma_y}, \quad (18)$$

where σ_y is the sample standard deviation of the observations. Other normalizations, such as the data range ($\max(y) - \min(y)$), may be appropriate in other applications, but the reported comparisons use the standard-deviation normalization. Lower NRMSE indicates a smaller forecast error relative to the variability of the series.